

Introducción a la Programación Honores

Quiz IV – N2

Por favor lea cuidadosamente las siguientes instrucciones:

- Este quiz es **individual** y su duración es hasta el final de la sesión de clase (10:50am).
- Está prohibida cualquier interacción con sus compañeros durante el examen.
- No se permite el uso de dispositivos electrónicos (celulares, relojes inteligentes, agendas electrónicas, computadores, etc.) ni de materiales adicionales como apuntes, libros o impresos.
- **Debe** escribir su respuesta de manera clara y legible, aplicando la sintaxis y los conceptos de Python estudiados en el curso. Se provee una cuadrícula para facilitar la indentación de su código; úsela de manera que quede claro en qué cuadro inicia cada instrucción. **Las respuestas ilegibles o inadecuadamente indentadas, pueden ser calificadas como incorrectas.** Puede usar lápiz, **pero únicamente se aceptarán reclamos sobre respuestas escritas en esfero.**
- Ante un intento de copia, el encargado podrá **retirarle el enunciado** y asignar una nota de **0.0** o iniciar un **proceso disciplinario**. Por favor, mantenga la mirada exclusivamente en su enunciado.

Compromiso de integridad:

Al proporcionar sus datos a continuación, usted confirma su compromiso de completar este examen de acuerdo con las normas de integridad académica de la Universidad de los Andes, incluyendo acatar el Reglamento General de Estudiantes de Pregrado y todas las disposiciones relacionadas con evitar el *Fraude académico*¹, establecidas en el Reglamento Disciplinario para Estudiantes de Pregrado.

Nombre: _____

Login: _____

Firma: _____

¹ En el artículo 4 del Reglamento Disciplinario para Estudiantes de Pregrado, se define el *Fraude académico* como “el comportamiento del estudiante que infringe las reglas de la Universidad o las reglas establecidas por el evaluador, en desarrollo de una actividad académica”.

Las dos preguntas se basan en **N2-PROY: CupiAdmisiones**. Aquí un recordatorio de la función que crea el diccionario de un candidato:

Nombre de la función		crear_candidato
Descripción de la función		Crea y retorna un diccionario que representa un candidato a ingresar a la universidad con todos sus atributos requeridos.
Parámetros	Tipo	Descripción
nombre	str	Nombre completo del candidato. Ejemplo: "Zutano Mengano".
doc_identidad	int	Número de documento de identidad. Ejemplo: 1023456789.
edad	int	Edad del candidato en años. Ejemplo: 18.
genero	str	Género del candidato. Ejemplo: "Masculino".
fecha_postulacion	str	Fecha de postulación en formato "YYYY-MM-DD". Ejemplo: "2025-06-21".
nacionalidad	str	Nacionalidad del candidato. Ejemplo: "Colombiana".
num_premios_academicos	int	Número de premios académicos obtenidos. Ejemplo: 3.
puntaje_saber11	int	Puntaje global obtenido en la Prueba Saber 11°. Ejemplo: 351.
carrera_aplicada	str	Carrera universitaria a la que aplica. Ejemplo: "Ingeniería Civil".
facultad_aplicada	str	Facultad correspondiente a la carrera. Ejemplo: "Ingeniería".
tiene_discapacidad	bool	Indica si el candidato tiene alguna discapacidad.
es_indigena	bool	Indica si el candidato pertenece a una comunidad indígena.
tiene_SISBEN	bool	Indica si el candidato está registrado en el SISBEN.
es_victima_conflicto	bool	Indica si el candidato es víctima del conflicto armado.
es_becado	bool	Indica si el candidato cuenta con alguna beca.
ingresos_nucleo_familiar	float	Ingresos mensuales del núcleo familiar. Ejemplo: 1500000.0
Retorno		
dict	Diccionario que representa al candidato, con las llaves coincidiendo con los nombres de los parámetros.	

Pregunta 1. Implemente la función `tiene_saber11_apto`.

Pregunta 2. Implemente la función `puntaje_para_beca_gobierno`.

Requerimientos para las preguntas 1 y 2:

1. DEBE usar condicionales y una variable de retorno.
2. NO puede usar funciones como `min()` o `max()`.
3. DEBE **usar todas las constantes** que se le proporcionan.

IMPORTANTE:

- Ya se provee *signatura* y *docstring*.
- No debe crear *docstests* ni aserciones.
- Debe implementar únicamente el cuerpo de las funciones.

Rúbrica de calificación tiene_saber11_apto:

Descripción	Cumple	Incumple
(5 pts) Usa correctamente una variable de retorno (se inicializa y retorna).	5	0
(5 pts) Obtiene correctamente la facultad y el puntaje del candidato.	5	0
(5 pts) Implementa con condicionales el caso Artes y Humanidades y Ciencias Sociales: 345	5	0
(5 pts) Implementa con condicionales el caso Derecho y Economía: 350 puntos.	5	0
(5 pts) Implementa con condicionales el caso Administración: 355 puntos.	5	0
(5 pts) Implementa con condicionales el caso Ciencias: 360 puntos.	5	0
(5 pts) Implementa con condicionales el caso Ingeniería: 370 puntos.	5	0
(5 pts) Implementa con condicionales el caso Medicina: 375 puntos.	5	0
(6 pts) Usa las constantes siguiendo la convención vista en el curso.	6	No usada = -1 c/u
(4 pts) El valor retornado corresponde al tipo de dato especificado en la signature.	4	0
Total	____ / 50	

Rúbrica de calificación puntaje_para_beca_gobierno:

Descripción	Cumple	Incumple
(5 pts) Usa correctamente una variable de retorno (se inicializa y retorna).	5	0
(10 pts) Determina si el candidato es apto, reutilizando una función preexistente.	10	0
(4 pts) Determina si el candidato es becado.	4	0
(4 pts) Implementa con condicionales el caso de nacionalidad colombiana.	4	0
(6 pts) Implementa con condicionales el caso de número de premios.	6	0
(4 pts) Implementa con condicionales el caso de tener SISBEN.	4	0
(4 pts) Implementa con condicionales el caso de ser víctima de conflicto.	4	0
(6 pts) Implementa con condicionales el caso de ingresos por núcleo familiar.	6	0
(3 pts) Usa las constantes siguiendo la convención vista en el curso.	3	No usada = -1 c/u
(4 pts) El valor retornado corresponde al tipo de dato especificado en la signature.	4	0
Total	____ / 50	

Observaciones (espacio para quien califica): _____

```
def tiene_saber11_apto(candidato: dict) -> bool:
    """
    Determina si el candidato tiene un puntaje Saber 11° igual o superior al mínimo requerido según la facultad a la
    que aplica. Los puntajes mínimos son determinados por el siguiente listado de facultades:
        - Artes y Humanidades ó Ciencias Sociales: 345 puntos.
        - Derecho ó Economía: 350 puntos.
        - Administración: 355 puntos.
        - Ciencias: 360 puntos.
        - Ingeniería: 370 puntos.
        - Medicina: 375 puntos.

    Args:
        candidato (dict): Diccionario que representa al candidato.

    Returns:
        bool: True si el puntaje Saber 11° es suficiente para la facultad correspondiente, False en caso contrario.
    """
    MINIMO_ARTES_Y_CIENCIAS_SOCIALES = 345
    MINIMO_DERECHO_Y_ECONOMIA = 350
    MINIMO_ADMINISTRACION = 355
    MINIMO_CIENCIAS = 360
    MINIMO_INGENIERIA = 370
    MINIMO_MEDICINA = 375
```



```
def puntaje_para_beca_gobierno(candidato: dict, max_ingreso_nucleo_familiar: float) -> float:
    """
    Calcula un puntaje para aspirar a una beca del gobierno colombiano. Hay dos casos generales:

    1. Si el candidato no tiene el puntaje Saber 11° mínimo requerido para ser admitido a la facultad que aplica
    o ya es becado, el puntaje es 0.0.

    2. Caso contrario, se suman 0.2 puntos por cada condición que se cumpla (inicia en 0.0):
        - Nacionalidad es "Colombiana".
        - Tiene más de 0 premios académicos.
        - Está registrado en el SISBEN.
        - Es víctima del conflicto.
        - Ingresos del núcleo familiar son menores o iguales al máximo permitido (max_ingreso_nucleo_familiar).

    Args:
        candidato (dict): Diccionario que representa al candidato.
        max_ingreso_nucleo_familiar (float): Valor máximo permitido de ingresos familiares mensuales.

    Returns:
        float: Puntaje entre 0.0 y 1.0 de elegibilidad para beca del gobierno colombiano, redondeado a 2 decimales.
    """
    NACIONALIDAD_COLOMBIANA = "Colombiana"
    PUNTOS_POR_CONDICION = 0.2
    MINIMO_PREMIOS = 0
```

